

DecodeLabs Internships 2026

DEVOPS PROJECT 3

CI/CD Pipeline Basics

Submitted By

Name: Jania Jose

Role: DevOps Intern

Department: B.Tech Computer Science and Engineering

College: Muthoot Institute of Technology and Science

Submitted To

DecodeLabs

Project Details

Project Number: Project 3

Project Title: CI/CD Pipeline Basics

Technology Used: Git,Github,Github Actions,WSL(Ubuntu)

Submission Date: 12-06-2026

Declaration

I hereby declare that this project report titled "**CI/CD Pipeline Basics**" has been completed by me as part of the DecodeLabs Industrial Training Program. The work presented in this report is based on my own practical implementation and learning experience in creating an automated CI/CD workflow using GitHub Actions. The project demonstrates the concepts of Continuous Integration, workflow automation, and pipeline execution in a DevOps environment.

Project 3: CI/CD Pipeline Basics – Introduction

In today's fast-paced software industry, organizations strive to deliver high-quality applications quickly and reliably. Traditional software development processes often involve manual testing, integration, and deployment activities, which can be time-consuming and prone to human error. To address these challenges, modern development teams adopt DevOps practices that emphasize automation, collaboration, and continuous improvement. Among the most important DevOps concepts are Continuous Integration (CI) and Continuous Deployment (CD), collectively known as CI/CD.

Continuous Integration is a software development practice in which developers frequently integrate code changes into a shared repository. Whenever changes are committed, automated processes are triggered to verify that the code functions correctly and does not introduce errors into the application. These automated checks may include compilation, testing, code quality analysis, and validation procedures. By identifying issues early in the development cycle, Continuous Integration helps improve software quality and reduces the effort required to resolve defects later.

Continuous Deployment extends the principles of Continuous Integration by automating the release and deployment process. Once code passes all required validation stages, it can be automatically deployed to testing, staging, or production environments. This automation eliminates many manual tasks associated with software delivery, leading to faster release cycles and more reliable deployments. Organizations that successfully implement CI/CD practices can deliver features and updates more frequently while maintaining high levels of quality and stability.

GitHub Actions is a powerful automation platform integrated directly into GitHub that enables developers to create and execute workflows based on repository events. Workflows are defined using YAML configuration files and can be triggered by actions such as code pushes, pull requests, issue creation, or scheduled events. GitHub Actions provides a flexible and scalable environment for implementing CI/CD pipelines without requiring additional infrastructure. As a result, it has become one of the most popular tools for automating software development workflows.

This project focuses on building a basic CI/CD pipeline using GitHub Actions. The workflow is configured to automatically execute whenever changes are pushed to the repository. Through this implementation, key concepts such as workflow creation, automation triggers, job execution, and pipeline monitoring are explored. The project demonstrates how repetitive tasks can be automated, reducing manual effort and improving development efficiency.

By completing this project, practical experience is gained in designing and managing automated workflows within a DevOps environment. The project highlights the importance of automation in modern software development and provides a foundation for more advanced topics such as automated testing, deployment strategies, infrastructure as code, container orchestration, and cloud-native DevOps practices. Understanding CI/CD pipelines is a critical step toward becoming a proficient DevOps engineer capable of building efficient and reliable software delivery systems.

Objective

The objective of this project is to understand the fundamentals of Continuous Integration and Continuous Deployment (CI/CD) by creating and executing an automated workflow using GitHub Actions. The project focuses on automating build and validation tasks whenever changes are pushed to a repository. Through this implementation, I gained practical experience in pipeline automation, workflow execution, and modern DevOps practices used to improve software delivery and reliability.

Step 1: Go to Your Repository

```
jania_jose_kannath@JANIAJOSE:~$ cd ~/Git_Project2
janias_jose_kannath@JANIAJOSE:~/Git_Project2$
```

Step 2: Create GitHub Actions Directory

```
janias_jose_kannath@JANIAJOSE:~$ cd ~/Git_Project2
janias_jose_kannath@JANIAJOSE:~/Git_Project2$ mkdir -p .github/workflows
```

Step 3: Create Workflow File

```
janias_jose_kannath@JANIAJOSE:~/Git_Project2$ nano .github/workflows/ci.yml
janias_jose_kannath@JANIAJOSE:~/Git_Project2$
```

Step 4: Verify Workflow File

```
janias_jose_kannath@JANIAJOSE:~/Git_Project2$ nano .github/workflows/ci.yml
janias_jose_kannath@JANIAJOSE:~/Git_Project2$ cat .github/workflows/ci.yml
name: CI Pipeline

on:
  push:
    branches:
      - main

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout Repository
        uses: actions/checkout@v4

      - name: Display Message
        run: echo "CI/CD Pipeline Executed Successfully"

      - name: Show Date
        run: date

      - name: List Files
        run: ls -la
```

Step 5: Add Files

```
janias_jose_kannath@JANIAJOSE:~/Git_Project2$ git add .
janias_jose_kannath@JANIAJOSE:~/Git_Project2$
```

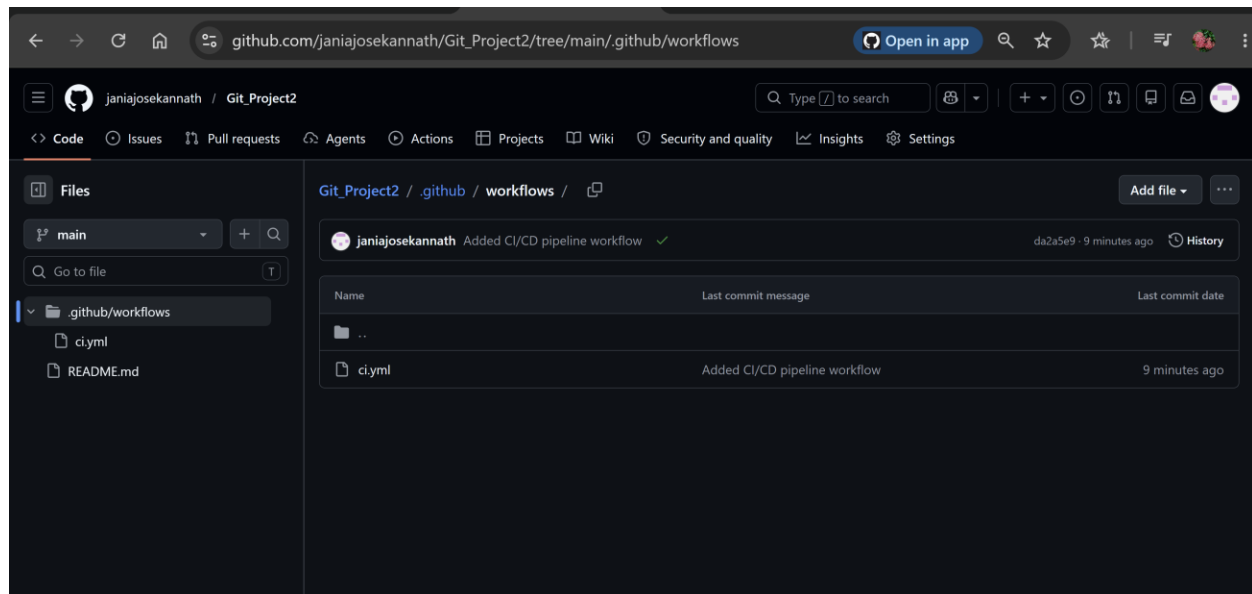
Step 6: Commit Workflow

```
jania_jose_kannath@JANIAJOSE:~/Git_Project2$ git commit -m "Added CI/CD pipeline workflow"
[main da2a5e9] Added CI/CD pipeline workflow
1 file changed, 23 insertions(+)
create mode 100644 .github/workflows/ci.yml
jania_jose_kannath@JANIAJOSE:~/Git_Project2$ |
```

Step 7: Push to GitHub

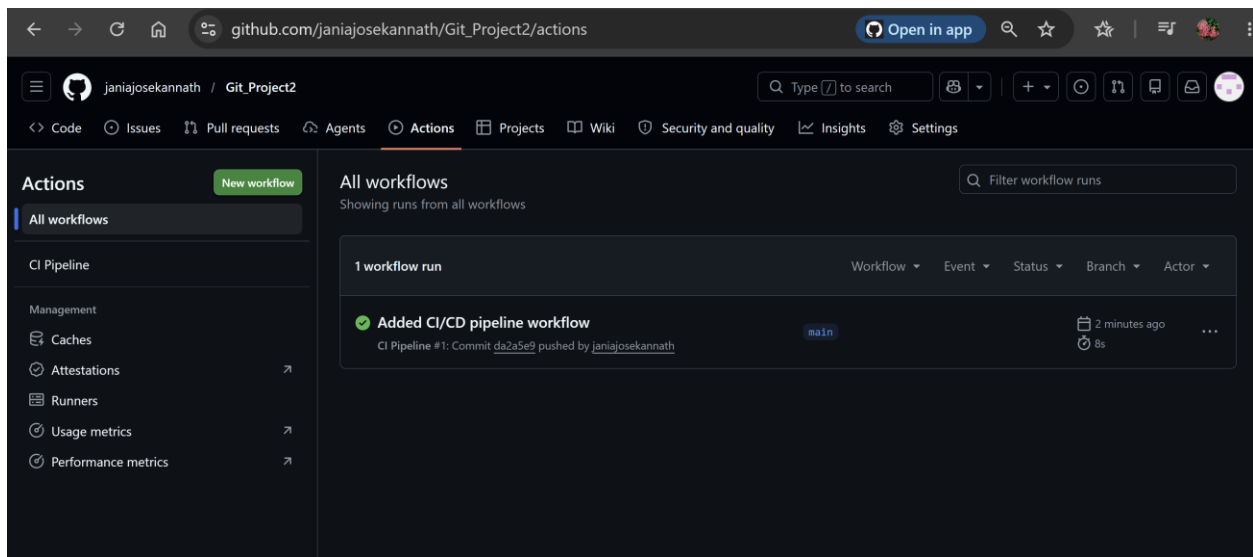
```
jania_jose_kannath@JANIAJOSE:~/Git_Project2$ git push -u origin main --force
Username for 'https://github.com': janiajosekannath
Password for 'https://janijosekannath@github.com':
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (5/5), 592 bytes | 148.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/janiajosekannath/Git_Project2.git
+ 361d066...da2a5e9 main -> main (forced update)
branch 'main' set up to track 'origin/main'.
jania_jose_kannath@JANIAJOSE:~/Git_Project2$ |
```

Step 8: Verify Pipeline Execution

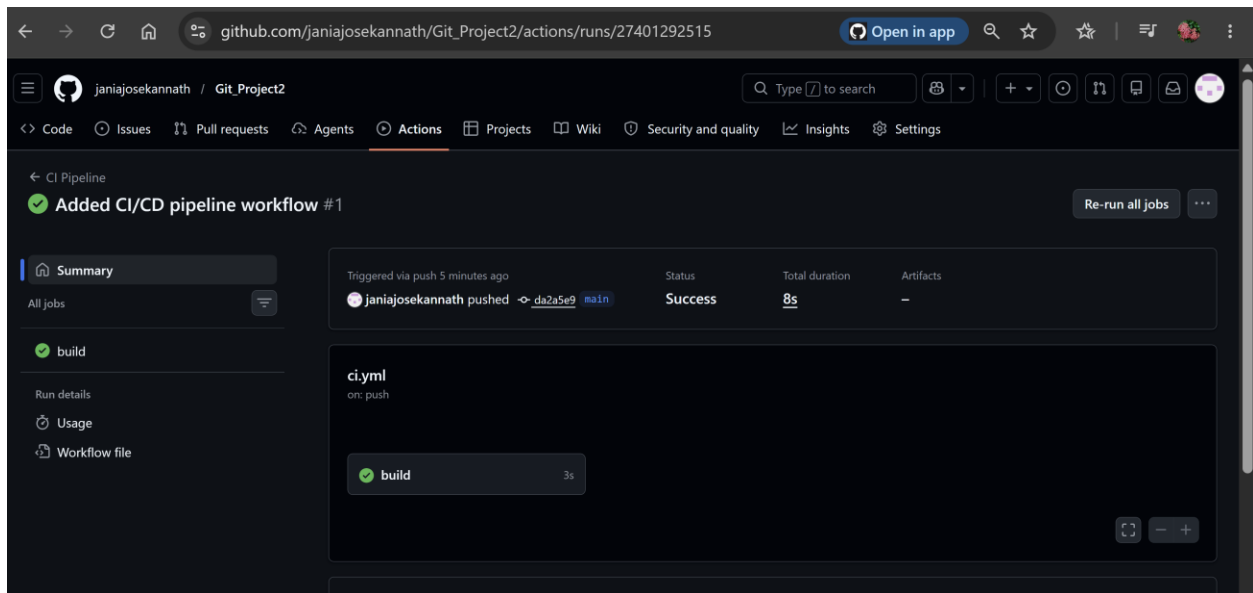


The screenshot shows the GitHub repository page for `Git_Project2` by user `janijosekannath`. The left sidebar shows the file structure with `.github/workflows/ci.yml` selected. The main content area displays the workflow execution history for `ci.yml`.

Name	Last commit message	Last commit date
..		
ci.yml	Added CI/CD pipeline workflow	9 minutes ago



Step 9: Open Workflow Details



GitHub Repository Link-

https://github.com/janiajosekannath/Git_Project2

Conclusion

In this project, I successfully implemented a basic CI/CD pipeline using GitHub Actions and integrated it with a GitHub repository. The workflow was configured to execute automatically whenever code changes were pushed to the repository, demonstrating the principles of Continuous Integration. Through this hands-on implementation, I learned how automated workflows help streamline software development, reduce manual effort, and improve the reliability of code delivery.

The project provided practical experience in creating workflow files, managing Git repositories, automating tasks, and monitoring pipeline execution through GitHub Actions. Overall, this project strengthened my understanding of CI/CD concepts and enhanced my knowledge of DevOps practices used in modern software development environments. It also highlighted the importance of automation in achieving faster, more efficient, and more reliable software delivery processes.